

Homework 1 Clustering and Regression

Instructions

Answer the questions and upload your answers to courseville. Answers can be in Thai or English. Answers can be either typed or handwritten and scanned. the assignment is divided into several small tasks. Each task is weighted equally (marked with **T**). For this assignment, each task is awarded 1 points. There are also optional tasks (marked with **OT**) counts for 0.5 points each.

Metrics

In a population where the amount of cats is equal to the amount of dogs. Considering the following classification results from a classifier.

Model A	Predicted dog	Predicted cat
Actual dog	30	20
Actual cat	10	40

T1. What is the accuracy of Model A?

Answer $(30+40) / (30+20+10+40) = 0.7$

T2. Consider cats as 'class 1' (positive) and dogs as 'class 0' (negative), calculate the precision, recall, and F1.

Answer Precision = $40 / (40+20) = 0.67$

Recall = $40 / (40+10) = 0.8$

F1 = $2 * 0.67 * 0.8 / (0.67 + 0.8) = 0.73$

T3. Consider class cat as 'class 0' and class dog as 'class 1', calculate the precision, recall, and F1.

Answer Precision = $30 / (30+10) = 0.75$

Recall = $30 / (30+20) = 0.6$

F1 = $2 * 0.75 * 0.6 / (0.75 + 0.6) = 0.67$

It is important to specify the 'positive' class when you calculate precision, recall, and F1. If there are more than two classes, it is usually done in a one- versus-all setting where one class is considered positive and the rest of the classes are considered negative.

T4. Now consider a lopsided population where there are 80% cats. What is the accuracy of Model A? Using dog as the positive class, what is the precision, recall, and F1? Explain how and why these numbers change (or does not change) from the previous questions.

Answer พิจารณาจาก Recall ของแมวที่มากกว่าสุนัข หมายความว่าถ้ามี ภาพแมวมา จะมีโอกาสตอบถูกว่าเป็น แมวมากกว่า มีภาพสุนัขแล้วตอบถูกว่าเป็นสุนัข -> อัตราการตอบชนิดสัตว์ถูกของแมวมักจะสูงกว่าสุนัข ดังนั้น ถ้าเพิ่ม population ของแมว หมายความว่า อัตราการตอบถูกโดยรวมจะมากขึ้น = **Accuracy increases**

Recall จะเทียบจาก Actual ของ class นั้นๆเท่านั้น (สนใจความแม่นยำของ class นั้นว่ามีการทายถูกแค่ไหน) ดังนั้นถ้ายังเป็น model เดิม Recall ก็จะไม่เปลี่ยนแปลง = **Recall is the same**

Precision จะเทียบจากการถูกทำนายเป็น class นั้นๆ (สนใจความแม่นยำการทายเป็น class นั้นๆ) ถ้าแมวมีย่านการทดสอบมากขึ้น หมายความว่า จำนวนการทายแมวว่าเป็นสุนัขก็จะมากขึ้น ทำให้ Precision ลดลง = **Precision Decrease**

F1 จากการที่ Recall เท่าเดิม แล้ว Decrease ลดลง ส่งผลให้ผลเฉลี่ยทาง harmonic ลดลง = **F1 Decrease**

OT1. Consider the equations for accuracy and F1

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ F1 &= \frac{2TP}{2TP + FP + FN} \end{aligned} \tag{1}$$

When will accuracy be equal, greater, or less than F1?

Answer จากสมการ ถ้า $TP = TN$ สมการ accuracy จะเหมือนกับ F1 ดังนั้น **F1 = Accuracy** เมื่อ **$TP = TN$**
แต่ถ้า TN เปลี่ยนแปลง F1 จะไม่มีเปลี่ยน เพราะ F1 ไม่ได้ขึ้นกับ TN ดังนั้นถ้า ค่าทุกอย่างคงที่ ยกเว้น TN เมื่อ
TN ลดลง -> Accuracy น้อยลงทำให้น้อยกว่า F1 ในทางกลับกันถ้า **TN เพิ่มขึ้น** -> Accuracy มากขึ้นทำให้
มากกว่า F1

Hello Clustering

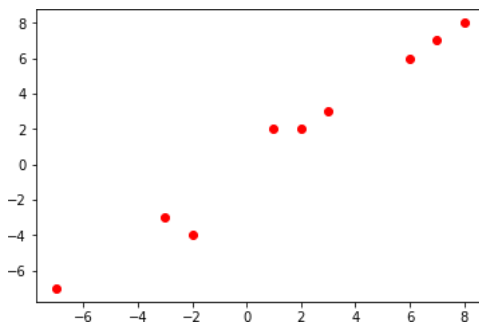
Recall from lecture that K-means has two main steps: the points assignment step, and the mean update step. After the initialization of the centroids, we assign each data point to a centroid. Then, each centroids are updated by re-estimating the means.

Concretely, if we are given N data points, $x_1, x_2, \dots, x_N$, and we would like to form K clusters. We do the following;

1. **Initialization:** Pick K random data points as K centroid locations $c_1, c_2, \dots, c_K$.
2. **Assign:** For each data point k , find the closest centroid. Assign that data point to the centroid. The distance used is typically Euclidean distance.
3. **Update:** For each centroid, calculate the mean from the data points assigned to it.
4. **Repeat:** repeat step 2 and 3 until the centroids stop changing (convergence).

Given the following data points in x-y coordinates (2 dimensional)

x	y
1	2
3	3
2	2
8	8
6	6
7	7
-3	-3
-2	-4
-7	-7



T5. If the starting points are $(3,3)$, $(2,2)$, and $(-3,-3)$. Describe each assign and update step. What are the points assigned? What are the updated centroids? You may do this calculation by hand or write a program to do it.

Answer

- หา data point ใกล้ centroid ไหนสุด

```

import numpy as np
import pandas as pd
coord = np.array([
    [1, 3, 2, 8, 6, 7, -3, -2, -7],
    [2, 3, 2, 8, 6, 7, -3, -4, -7]
]).T
starting = np.array([
    [3, 2, -3],
    [3, 2, -3]
])

dx = coord[:, 0:1] - starting[0:1]
dy = coord[:, 1:2] - starting[1:2]
eu_dist = np.sqrt(dx**2 + dy**2)
eu_dist_frame = pd.DataFrame(eu_dist, columns=[f"start_point_{start}" for start in starting.T])

print(f"euclidean distance: data point - start point")
eu_dist_frame

```

euclidean distance: data point - random point

	start_point [3 3]	start_point [2 2]	start_point [-3 -3]
0	2.236068	1.000000	6.403124
1	0.000000	1.414214	8.485281
2	1.414214	0.000000	7.071068
3	7.071068	8.485281	15.556349
4	4.242641	5.656854	12.727922
5	5.656854	7.071068	14.142136
6	8.485281	7.071068	0.000000
7	8.602325	7.211103	1.414214
8	14.142136	12.727922	5.656854

```

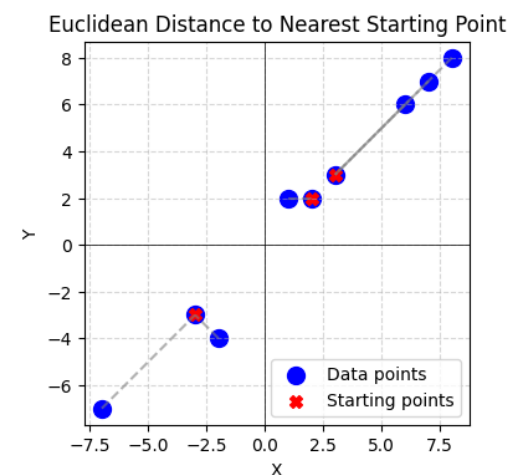
dist_min_idx = eu_dist.argmin(axis=1)
for idx, min_idx in enumerate(dist_min_idx):
    print(f"Point {coord[idx]} is closest to point {starting.T[min_idx]}")

```

```

Point [1 2] is closest to point [2 2]
Point [3 3] is closest to point [3 3]
Point [2 2] is closest to point [2 2]
Point [8 8] is closest to point [3 3]
Point [6 6] is closest to point [3 3]
Point [7 7] is closest to point [3 3]
Point [-3 -3] is closest to point [-3 -3]
Point [-2 -4] is closest to point [-3 -3]
Point [-7 -7] is closest to point [-3 -3]

```



- Updated centroid

```
k = starting.shape[1]
new_centroids = np.zeros((2, k))

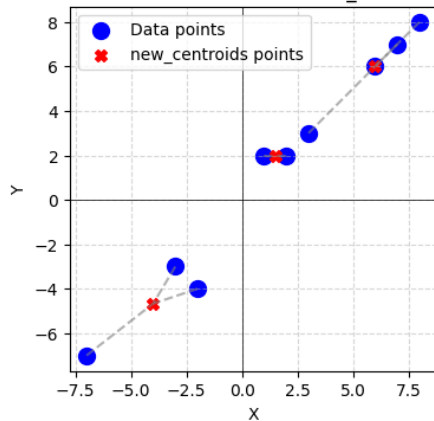
for j in range(k):
    assigned_points = coor[dist_min_idx == j]

    if len(assigned_points) > 0:
        new_centroids[:, j] = assigned_points.mean(axis=0)
    else:
        new_centroids[:, j] = starting[:, j]
print("New centroids:")
new_centroids.T
```

New centroids:

```
array([[ 6.         ,  6.         ],
       [ 1.5        ,  2.         ],
       [-4.         , -4.66666667]])
```

Euclidean Distance to Nearest new_centroids Point



T6. If the starting points are (-3,-3), (2,2), and (-7,-7), what happens?

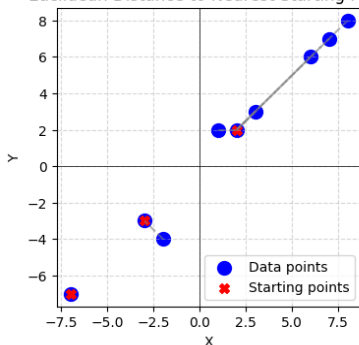
Answer Random point ที่ต่างกันจะทำให้ผลลัพธ์การจัดกลุ่มที่ต่างกันออกไป

```
Point [1 2] is closest to point [2 2]
Point [3 3] is closest to point [2 2]
Point [2 2] is closest to point [2 2]
Point [8 8] is closest to point [2 2]
Point [6 6] is closest to point [2 2]
Point [7 7] is closest to point [2 2]
Point [-3 -3] is closest to point [-3 -3]
Point [-2 -4] is closest to point [-3 -3]
Point [-7 -7] is closest to point [-7 -7]
```

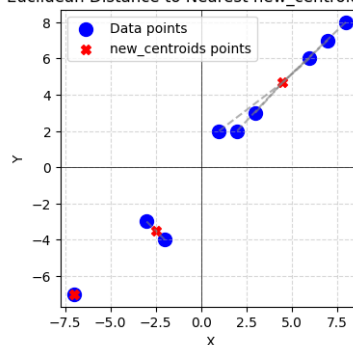
New centroids:

```
array([[ -2.5        ,  -3.5        ],
       [  4.5        ,  4.66666667],
       [ -7.         ,  -7.         ]])
```

Euclidean Distance to Nearest Starting Point



Euclidean Distance to Nearest new_centroids Point



T7. Between the two starting set of points in the previous two questions, which one do you think is better? How would you measure the 'goodness' quality of a set of starting points?

Answer ชุดที่2ดีกว่าเพราะหลัง update centroid ของชุดแรกแล้วจะมีจุดหนึ่งที่อยู่ผิดกลุ่มทำให้เกิดการลู่เข้าที่ช้าขึ้น แต่ในชุดที่2หลัง update centroid แล้วได้คำตอบเลย

In general, it is important to try different sets of starting points when doing k-means.

OT2. What would be the best K for this question? Describe your reasoning.

Answer 4 เพราะแต่ละกลุ่มมีระยะห่างที่ค่อนข้างคงที่ ซึ่งมีทั้งหมด4กลุ่มที่ติดเป็นแบบนั้น จึงคิดว่า K ควรเท่ากับ 4

My heart will go on¹



In this part of the exercise we will work on the Titanic dataset provided by Kaggle. The Titanic dataset contains information of the passengers boarding the Titanic on its final voyage. We will work on predicting whether a given passenger will survive the trip.

Let's launch Jupyter and start coding!

We start by importing the data using Pandas

```
train_url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/train.csv" train =  
pd.read_csv(train_url) #training set
```

```
test_url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/test.csv" test =  
pd.read_csv(test_url) #test set
```

Both train and test are dataframes. Use the function `train.head()` and `train.tail()` to explore the data. What do you see?

Use the function `describe()` to get a better understanding of the data. You can read the meaning of the data fields at <https://www.kaggle.com/c/titanic/data>

¹Many parts of this exercise are adapted from Kaggle Python Tutorial on Machine Learning

Looking at the data, you will notice a lot of missing values. For example, some age is NaN. This is normal for real world data to have some missing values. There are several ways to handle missing values. The simplest is to throw away any rows that have missing values. However, this usually reduce the amount of training data you have. Another method is to guess what the missing value should be. The simplest guess is to use the Median or Mode of the data. For this exercise we will proceed with this.

T8. What is the median age of the training set? You can easily modify the age in the dataframe by

Answer 28.0

```
train["Age"] = train["Age"].fillna(train["Age"].median())
```

Note that you need to modify the code above a bit to fill with mode() because mode() returns a series rather than a single value.

T9. Some fields like 'Embarked' are categorical. They need to be converted to numbers first. We will represent S with 0, C with 1, and Q with 2. What is the mode of Embarked? Fill the missing values with the mode. You can set the value of Embarked easily with the following command.

Answer 0

```
train.loc[train["Embarked"] == "S", "Embarked"] = 0
```

Do the same for Sex.

T10. Write a logistic linear regression classifier using gradient descent as learned in class. Use PClass, Sex, Age, and Embarked as input features. You can extract the features from Pandas to Numpy by

Answer

```
def preprocess(df):
    df.loc[df["Embarked"] == "S", "Embarked"] = 0
    df.loc[df["Embarked"] == "C", "Embarked"] = 1
    df.loc[df["Embarked"] == "Q", "Embarked"] = 2
    df.loc[df["Sex"] == "male", "Sex"] = 0
    df.loc[df["Sex"] == "female", "Sex"] = 1
    df["Sex"] = df["Sex"].astype(int)
    return df

train_pre = preprocess(train)
test_pre = preprocess(test)

age_median = train_pre["Age"].median()
embarked_mode = train_pre["Embarked"].mode().iloc[0]

train_pre["Age"] = train_pre["Age"].fillna(age_median)
train_pre["Embarked"] = train_pre["Embarked"].fillna(embarked_mode)
data = np.array(train_pre[["Pclass", "Sex", "Age", "Embarked"]].values, dtype = float)

test_pre["Age"] = test_pre["Age"].fillna(age_median)
test_pre["Embarked"] = test_pre["Embarked"].fillna(embarked_mode)
test_data = np.array(test_pre[["Pclass", "Sex", "Age", "Embarked"]].values, dtype=float)

y = np.array(train["Survived"].values, dtype=float)

def linear_regression_gradient_descent(X, y, learning_rate=0.01, iterations=1000):
    m = len(y)
    X = np.c_[np.ones(m), X]

    n_features = X.shape[1]
    theta = np.zeros(n_features)

    for i in range(iterations):
        h = X.dot(theta)
        gradient = (1/m) * X.T.dot(h - y)
        theta = theta - learning_rate * gradient

    return theta
```



```
data = np.array(train[["PClass","Sex","Age","Embarked"]].values)
```

Check the datatype of each values in data, does it make sense? You can force the data to be of any datatype by using the command

```
data = np.array(train[["PClass","Sex","Age","Embarked"]].values, dtype = float)
```

When you evaluate the trained model on the test set, you will need to make a final decision. Since logistic linear regression outputs a score between 0 and 1, you will need to decide whether a score of 0.3 (or any other number) means the passenger survive or not. For now, we will say if the score is greater than or equal to 0.5, the passenger survives. If the score is lower than 0.5 the passenger will be dead. This process is often called 'Thresholding.' We will talk more about this process later in class.

To evaluate your results, we will use Kaggle. Kaggle is a website that hosts many machine learning competitions. Many companies put up their data as a problem for anyone to participate. If you are looking for a task for your course project, Kaggle might be a good place to start. You will need to make sure that your output is in line with the submission requirements of Kaggle: a csv file with exactly 418 entries and two columns: PassengerId and Survived. Then, use the code provided to make a new data frame using DataFrame(), and create a csv file using to_csv() method from Pandas.

To submit your prediction, you must first sign-up for an account on Kaggle. com. Click participate to the competition at <https://www.kaggle.com/c/titanic/> then submit your csv file for the score.

The output file should have two columns: the passengerId and a 0,1 decision (0 for dead, 1 for survive). As shown below:

PassengerId,Survived

892,0

893,1

894,0

T11. Submit a screenshot of your submission (with the scores). Upload your code to courseville.

Answer

- theta: [0.38383838 -0.15746036 0.2344979 -0.06576928 0.0312026]
- Training MSE: 0.14492573766134814
Training Accuracy: 0.7991021324354658



T10_submission.csv
Complete · 3m ago

0.76076

T12. Try adding some higher order features to your training ($x^2, x_1x_2, \dots$). Does this model has better **accuracy on the training set**? How does it perform on the **test set**?

Answer

- theta: [0.38383838 -0.14127159 0.27975698 -0.22210426 0.04887783 0.12379878 0.0790349
-0.1439937 0.0426937 0.03169695 -0.04106522]
- Training MSE: 0.1389825890288719
Training Accuracy: 0.819304152637486
- ทั้ง Training และ Test set มี accuracy ที่ดีขึ้น



T12_submission.csv
Complete · 3m ago

0.77990

T13. What happens if you reduce the amount of features to just Sex and Age?

Answer

- theta: [0.38383838 0.26341541 -0.01018773]
- Training MSE: 0.16657939408141703
Training Accuracy: 0.7867564534231201
- Training set มี accuracy ที่ต่ำลง แต่ Test set มีผลลัพธ์ที่ดีขึ้นเล็กน้อย



T13_submission.csv
Complete · 2m ago

0.76555

All Results:



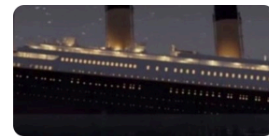
KAGGLE · GETTING STARTED PREDICTION COMPETITION · ONGOING

Submit Prediction

...

Titanic - Machine Learning from Disaster

Start here! Predict survival on the Titanic and get familiar with ML basics



Overview Data Code Models Discussion Leaderboard Rules Team Submissions

Submissions

All

Successful

Errors

Recent ▾

Submission and Description

Public Score ⓘ



T13_submission.csv
Complete · now

0.76555



T12_submission.csv
Complete · 14s ago

0.77990



T10_submission.csv
Complete · 24s ago

0.76076

OT3. We want to show that matrix inversion yields the same answer as the gradient descent method.

Answer

```
m = len(y)
X = np.c_[np.ones(m), data_scaled]
matrix_inversion = np.linalg.inv(X.T.dot(X))
theta_normal = matrix_inversion.dot(X.T).dot(y)
```

OT4. Now try using matrix inversion instead. However Are the weights learned from the two methods similar? Report the Mean Squared Errors (MSE) of the difference between the two weights.

Answer

- Theta from Normal Equation: [0.38383838 -0.15746036 0.2344979 -0.06576928 0.0312026]
Theta from Gradient Descent: [0.38383838 -0.15746036 0.2344979 -0.06576928 0.0312026]
- Training MSE from Normal Equation: 0.14492573766134814
Training MSE from Gradient Descent: 0.14492573766134814
- ทั้ง2วิธีให้ผลลัพธ์ Theta และ MSE ที่เหมือนกัน (สามารถดูได้ใน code)

[Optional] Fun with matrix algebra

Prove the following statements. All of them can be solved by first expanding out the matrix notation as a combination of their elements, and then use the definitions of trace and matrix derivatives to help finish the proof. For example, the (i, j) element of $Y = AB$ is $Y_{i,j} = \sum_m A_{i,m} B_{m,j}$.

OT5. $\nabla_A \text{tr} AB = B^T$

OT6. $\nabla_{A^T} f(A) = (\nabla_A f(A))^T$

OT7. $\nabla_A \text{tr} ABA^T C = CAB + C^T AB^T$

Hint: Try first solving the easier equation of $\nabla_A \text{tr} BAC = (CB)^T = B^T C^T$

import Data

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: train_url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/train.csv"
train = pd.read_csv(train_url)
test_url = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/test.csv"
test = pd.read_csv(test_url)
```

Data preprocessing

```
In [3]: def standardize(X):
        """
        Standardize features to have mean=0 and std=1
        Formula: X_scaled = (X - mean) / std
        """
        mean = np.mean(X, axis=0)
        std = np.std(X, axis=0)

        # ป้องกันการหารด้วย 0
        std[std == 0] = 1

        X_scaled = (X - mean) / std
        return X_scaled, mean, std
```

```
In [4]: def preprocess(df):
        df.loc[df["Embarked"] == "S", "Embarked"] = 0
        df.loc[df["Embarked"] == "C", "Embarked"] = 1
        df.loc[df["Embarked"] == "Q", "Embarked"] = 2
        df.loc[df["Sex"] == "male", "Sex"] = 0
        df.loc[df["Sex"] == "female", "Sex"] = 1
        df["Sex"] = df["Sex"].astype(int)
        return df
```

```
In [5]: train_pre = preprocess(train)
test_pre = preprocess(test)

age_median = train_pre["Age"].median()
embarked_mode = train_pre["Embarked"].mode().iloc[0]
print("Age median:", age_median)
print("Embarked mode:", embarked_mode)

train_pre["Age"] = train_pre["Age"].fillna(age_median)
train_pre["Embarked"] = train_pre["Embarked"].fillna(embarked_mode)
data = np.array(train_pre[["Pclass", "Sex", "Age", "Embarked"]].values, dtype = object)

test_pre["Age"] = test_pre["Age"].fillna(age_median)
test_pre["Embarked"] = test_pre["Embarked"].fillna(embarked_mode)
```

```
test_data = np.array(test_pre[["Pclass", "Sex", "Age", "Embarked"]].values,
y = np.array(train["Survived"].values, dtype=float)
```

Age median: 28.0

Embarked mode: 0

```
/tmp/ipykernel_45856/1591505398.py:10: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
train_pre["Embarked"] = train_pre["Embarked"].fillna(embarked_mode)
/tmp/ipykernel_45856/1591505398.py:14: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
test_pre["Embarked"] = test_pre["Embarked"].fillna(embarked_mode)
```

Train & Test Function

```
In [6]: def calculate_mse(y_predict, y):
        m = len(y)
        mse = (1/m) * np.sum((y_predict - y)**2)
        return mse

        def calculate_accuracy(y_predict, y):
            predictions_binary = (y_predict >= 0.5).astype(int)
            accuracy = np.mean(predictions_binary == y)
            return accuracy
```

```
In [7]: def linear_regression_gradient_descent(X, y, learning_rate=0.01, iterations=
        m = len(y)
        X = np.c_[np.ones(m), X]

        n_features = X.shape[1]
        theta = np.zeros(n_features)

        for i in range(iterations):
            h = X.dot(theta)
            gradient = (1/m) * X.T.dot(h - y)
            theta = theta - learning_rate * gradient

        return theta
```

```
In [8]: def predict(X, theta):
        m = X.shape[0]
        X = np.c_[np.ones(m), X]
        predictions = X.dot(theta)
        return predictions

        def convert_to_binary(y):
            return (y >= 0.5).astype(int)
```

T10

```
In [9]: data_scaled, train_mean, train_std = standardize(data)
        theta = linear_regression_gradient_descent(data_scaled, y, learning_rate=0.001)
        y_predict = predict(data_scaled, theta)
        train_mse = calculate_mse(y_predict, y)
        train_accuracy = calculate_accuracy(convert_to_binary(y_predict), y)
        print("theta:", theta)
        print(f"Training MSE: {train_mse}")
        print(f"Training Accuracy: {train_accuracy}")
```

```
theta: [ 0.38383838 -0.15746036  0.2344979  -0.06576928  0.0312026 ]
Training MSE: 0.14492573766134814
Training Accuracy: 0.7991021324354658
```

```
In [10]: test_data_scaled = (test_data - train_mean) / train_std
        predictions = predict(test_data_scaled, theta)
```

```
In [11]: submission = pd.DataFrame({
        'PassengerId': test['PassengerId'],
        'Survived': convert_to_binary(predictions)
    })
        submission.to_csv('T10_submission.csv', index=False)
```

T12

```
In [12]: def preprocess_high_order(data_scaled):
        data_high_order = np.array(data_scaled)
        x4 = data_high_order[:, 2] ** 2
        x5 = data_high_order[:, 1] * data_high_order[:, 2]
        x6 = data_high_order[:, 1] * data_high_order[:, 0]
        x7 = data_high_order[:, 1] * data_high_order[:, 3]
        x8 = data_high_order[:, 2] * data_high_order[:, 0]
        x9 = data_high_order[:, 2] * data_high_order[:, 3]
        data_high_order = np.c_[data_high_order, x4, x5, x6, x7, x8, x9]
        return data_high_order
```

```
In [13]: data_high_order = preprocess_high_order(data)
        data_high_order, train_mean_high_order, train_std_high_order = standardize(data_high_order)
        theta_high_order = linear_regression_gradient_descent(data_high_order, y, learning_rate=0.001)
        y_predict_high_order = predict(data_high_order, theta_high_order)
        train_high_order_mse = calculate_mse(y_predict_high_order, y)
        train_high_order_accuracy = calculate_accuracy(convert_to_binary(y_predict_high_order), y)
        print("theta:", theta_high_order)
        print(f"Training MSE: {train_high_order_mse}")
        print(f"Training Accuracy: {train_high_order_accuracy}")
```

```
theta: [ 0.38383838 -0.14127159  0.27975698 -0.22210426  0.04887783  0.12379
878
 0.0790349 -0.1439937  0.0426937  0.03169695 -0.04106522]
Training MSE: 0.1389825890288719
Training Accuracy: 0.819304152637486
```

```
In [14]: test_high_order = preprocess_high_order(test_data)
test_data_scaled = (test_high_order - train_mean_high_order) / train_std_high_order
predictions_high_order = predict(test_data_scaled, theta_high_order)
```

```
In [15]: submission = pd.DataFrame({
    'PassengerId': test['PassengerId'],
    'Survived': convert_to_binary(predictions_high_order)
})
submission.to_csv('T12_submission.csv', index=False)
```

T13

```
In [16]: data_low_feat = np.array(train_pre[["Sex", "Age"]].values, dtype = float)
data_low_feat_scaled, train_low_feat_mean, train_low_feat_std = standardize(
theta_low_feat = linear_regression_gradient_descent(data_low_feat_scaled, y,
y_predict_low_feat = predict(data_low_feat_scaled, theta_low_feat)
train_mse_low_feat = calculate_mse(y_predict_low_feat, y)
train_accuracy_low_feat = calculate_accuracy(convert_to_binary(y_predict_low_feat), y)
print("theta:", theta_low_feat)
print(f"Training MSE: {train_mse_low_feat}")
print(f"Training Accuracy: {train_accuracy_low_feat}")
```

```
theta: [ 0.38383838  0.26341541 -0.01018773]
Training MSE: 0.16657939408141703
Training Accuracy: 0.7867564534231201
```

```
In [17]: test_data_low_feat = np.array(test_pre[["Sex", "Age"]].values, dtype=float)
test_data_low_feat_scaled = (test_data_low_feat - train_low_feat_mean) / train_low_feat_std
predictions_low_feat = predict(test_data_low_feat_scaled, theta_low_feat)
```

```
In [18]: submission = pd.DataFrame({
    'PassengerId': test['PassengerId'],
    'Survived': convert_to_binary(predictions_low_feat)
})
submission.to_csv('T13_submission.csv', index=False)
```

Matrix Inversion

```
In [19]: m = len(y)
X = np.c_[np.ones(m), data_scaled]
matrix_inversion = np.linalg.inv(X.T.dot(X))
theta_normal = matrix_inversion.dot(X.T).dot(y)
```

```
In [20]: for row in matrix_inversion.tolist():
    print(row)
```

```
[0.001122334455667789, 1.3336326605640432e-20, -2.44846666182638e-19, -4.362
348113840282e-19, 2.1860174916300365e-19]
[1.3336326605640392e-20, 0.001312565320773622, 0.00022021794148560338, 0.000
4632660556023183, -8.141142949442723e-05]
[-2.4484666618263807e-19, 0.00022021794148560336, 0.0011821914183460767, 0.0
0016946032841411423, -0.00014631861771918826]
[-4.362348113840283e-19, 0.00046326605560231826, 0.00016946032841411423, 0.0
01293285186081991, -2.907283520233438e-05]
[2.186017491630037e-19, -8.141142949442722e-05, -0.00014631861771918824, -2.
9072835202334368e-05, 0.0011428449292687458]
```

```
In [21]: print("Theta from Normal Equation:")
print(theta_normal)
print("Theta from Gradient Descent:")
print(theta)
```

```
Theta from Normal Equation:
[ 0.38383838 -0.15746036  0.2344979  -0.06576928  0.0312026 ]
Theta from Gradient Descent:
[ 0.38383838 -0.15746036  0.2344979  -0.06576928  0.0312026 ]
```

```
In [22]: y_predict_matrix_inversion = predict(data_scaled, theta_normal)
train_mse_matrix_inversion = calculate_mse(y_predict_matrix_inversion, y)
train_accuracy_matrix_inversion = calculate_accuracy(convert_to_binary(y_pre
print("theta normal:", theta_normal)
print(f"Training MSE: {train_mse_matrix_inversion}")
print(f"Training Accuracy: {train_accuracy_matrix_inversion}")
```

```
theta normal: [ 0.38383838 -0.15746036  0.2344979  -0.06576928  0.0312026 ]
Training MSE: 0.14492573766134814
Training Accuracy: 0.7991021324354658
```